Assignment 6: Visitor Pattern

Yerzhan Aitimov | SE-2526

Repository: https://github.com/TwoThousand07/SDP_Assignment6

1. Purpose and design

The application models an in-memory hierarchical file system. A file stores its name and non-negative byte size. A directory stores named elements and supports arbitrary nesting. The Element interface defines the shared element contract. AbstractElement holds the common name and parent reference; FileNode and DirectoryNode provide the concrete element types.

Three independent visitors operate on the same tree:

SizeVisitor calculates the total byte size of a subtree. Each file contributes its size once; directories contribute no size of their own.

SearchVisitor combines a filename predicate with an inclusive minimum-size threshold. It returns matching paths relative to the selected root, separated by / and sorted lexicographically.

StatisticsVisitor counts files and directories and reports maximum depth. A selected directory root is included in the directory count at depth zero.

Tree integrity is protected by validating names and file sizes, rejecting duplicate sibling names, multiple parents, and cycles, and exposing child elements as an immutable snapshot. Removing a child clears its parent reference.

2. Double dispatch and traversal

Each concrete node implements accept(visitor) by calling the matching overloaded visit method with itself. Java selects the overload from the node's compile-time type, completing double dispatch without instanceof checks, reflection, or casts. The Visitor interface declares one visit method for each concrete element type.

Each visitor owns its recursive traversal, keeping operation logic out of the element classes. Public operations reset their state before each traversal, so visitor instances can be reused safely. This choice keeps the element model small while allowing each operation to manage its own path or counters.

3. Clean Code practices

1. Intent-revealing names: calculate, search, and collect state the operation performed by each visitor method.

2. Single responsibility: each visitor implements one operation, while DirectoryNode owns tree mutation rules.

3. Encapsulation: getChildren returns an immutable snapshot, so callers cannot edit the internal child list.

4. Guard clauses: constructors and addChild reject invalid input before it changes the tree.

5. Deterministic output: SearchVisitor sorts root-relative paths and uses a documented path format.

4. Extensibility trade-off

Adding an operation requires a new visitor and implementations of its two visit methods. Adding an element type requires a new Visitor overload and an accept implementation, then every existing visitor must decide how to handle it. The compiler helps locate visitors that need updating. Visitors own traversal, so a short directory-walking loop is repeated across operations.

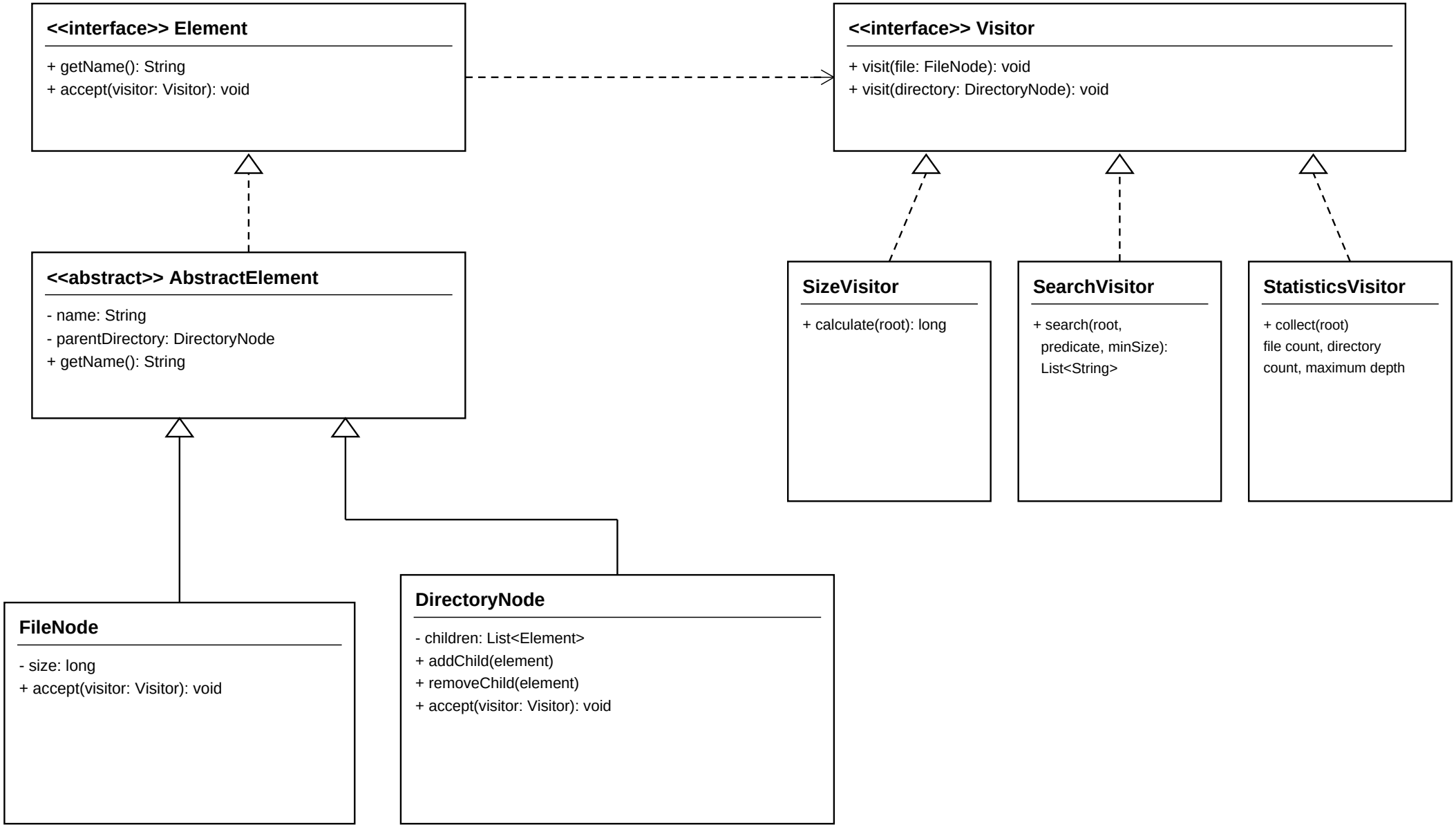
5. Verification and sample run

Run the automated suite from the project root with `mvn test`. The five JUnit classes are VisitorDispatchTest, SizeVisitorTest, SearchVisitorTest, StatisticsVisitorTest, and DirectoryNodeTest. Together they cover dispatch, nested size totals, combined search filters, sorted relative paths, statistics, empty and no-match cases, repeat executions, and invalid tree mutations.

Recorded result for this version: 14 tests, 0 failures, 0 errors, 0 skipped; BUILD SUCCESS.

Demonstration output

```
Total size: 2910 bytes
Matches:
assets/changelog.txt
docs/readme.txt
Files: 5
Directories: 4
Maximum depth: 3
```

Run the demonstration after compiling with `mvn -q -DskipTests compile`, then `java -cp target/classes edu.astana.visitorfs.App`.

UML class diagram